Generative AI Basics: GPTs and Text Generation

This notebook explores the fundamentals of Generative AI, focusing on the architecture and functionality of Generative Pre-trained Transformers (GPTs). We will implement a basic text generation model using Python and discuss practical applications and ethical considerations.

1. Setup and Imports

```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import os
import requests # For downloading dataset
import re # For text cleaning
from collections import Counter

print(f"TensorFlow Version: {tf.__version__}")
```

2. Introduction to Generative AI and GPTs

Introduction to Generative AI

Generative Artificial Intelligence refers to AI systems capable of generating new content, such as text, images, audio, or video, that is similar to human-created output but is unique. Unlike discriminative models that classify or predict based on input, generative models learn the underlying patterns and distribution of the training data to create novel samples.

Its significance lies in its potential to automate creative tasks, enhance productivity, and enable new forms of human-computer interaction across various domains, from content creation to drug discovery.

Exploring Generative Pre-trained Transformers (GPTs)

Generative Pre-trained Transformers (GPTs) are a family of powerful large language models (LLMs) developed by OpenAI. They are particularly renowned for their ability to generate human-like text across a wide range of tasks.

Model Architecture: The Transformer

At the core of GPTs is the **Transformer** architecture, introduced by Vaswani et al. in 2017. Transformers revolutionized sequence modeling by abandoning recurrent (RNNs, LSTMs) and convolutional (CNNs) structures in favor of **attention mechanisms**.

Key components of the Transformer architecture (relevant to GPTs):

- **Encoder-Decoder Structure (Modified for GPTs)**: The original Transformer has an encoder-decoder architecture. GPTs, however, are *decoder-only* transformers. This means they are designed purely for generating sequences, predicting the next token based on all previously generated tokens.
- **Self-Attention (Multi-Head Attention)**: This mechanism allows the model to weigh the importance of different words in the input sequence when processing each word. In GPTs, it's a *masked* self-attention, meaning a token can only attend to previous tokens in the sequence, preserving the autoregressive property essential for generation.
- **Feed-Forward Networks**: After the attention layers, each position in the sequence passes through an identical, independent feed-forward neural network.
- **Positional Encoding**: Since Transformers do not inherently process sequences in order, positional encodings are added to the input embeddings to inject information about the relative or absolute position of tokens in the sequence.

How GPTs Generate Text

GPTs generate text in an autoregressive manner, one token at a time:

1. **Tokenization**: The input text (seed or prompt) is first broken down into numerical tokens (subword units). GPT models use a vocabulary of these tokens.
2. **Input Embedding**: Each token is converted into a high-dimensional vector (embedding) that captures its semantic meaning.
3. **Positional Encoding**: Positional information is added to these embeddings.
4. **Transformer Blocks**: The combined embeddings pass through multiple layers of the decoder-only Transformer blocks, which process the sequence using masked self-attention and feed-forward networks.
5. **Probability Distribution**: The final layer outputs a probability distribution over the entire vocabulary for the *next* token in the sequence.
6. **Token Sampling**: The model samples a token from this probability distribution. Common sampling strategies include:
    - **Greedy Sampling**: Always pick the token with the highest probability.
    - **Beam Search**: Keep track of multiple high-probability sequences simultaneously.
    - **Temperature Sampling**: Introduce randomness by adjusting the `temperature` parameter; higher temperature leads to more diverse (but potentially less coherent) output.

- **Top-K or Top-P (Nucleus) Sampling**: Sample from the top K most probable tokens, or from the smallest set of tokens whose cumulative probability exceeds P.

7. **Sequence Generation**: The sampled token is appended to the input sequence, and the process repeats until a stop condition is met (e.g., maximum length reached, end-of-text token generated).

3. Dataset Preparation

We will use a small text file from Project Gutenberg for a simple character-level text generation example. This approach simplifies tokenization and allows for a basic RNN/LSTM model to learn sequential patterns.

```
# Download a text file (e.g., Shakespeare's Sonnets from Project Gutenberg)
url = "https://www.gutenberg.org/files/100/100-0.txt"
file_path = tf.keras.utils.get_file("sonnets.txt", url)

# Read and decode the text
text = open(file_path, 'rb').read().decode(encoding='utf-8')

# Simple text cleaning (remove header/footer, lowercasing, remove extra symbols)
start_marker = "*** START OF THE PROJECT GUTENBERG EBOOK THE SONNETS, BY WILLIAM SH
end_marker = "*** END OF THE PROJECT GUTENBERG EBOOK THE SONNETS, BY WILLIAM SHAKES

start_idx = text.find(start_marker)
end_idx = text.find(end_marker)

if start_idx != -1 and end_idx != -1:
    text = text[start_idx + len(start_marker):end_idx]

text = text.lower()
text = re.sub(r'[^a-z0-9\s.!?,-]', '', text) # Keep basic punctuation
text = re.sub(r'\s+', ' ', text).strip() # Replace multiple spaces with a single sp

print(f"Length of text: {len(text)} characters")
print("\nFirst 500 characters:\n")
print(text[:500])
```

Create Character Mappings

For a character-level model, we need to map each unique character to an integer and vice-versa.

```
vocab = sorted(list(set(text)))
char_to_int = {char: i for i, char in enumerate(vocab)}
int_to_char = {i: char for i, char in enumerate(vocab)}

print(f"Vocabulary size: {len(vocab)}")
print(f"Vocabulary: {vocab}")
```

Prepare Training Sequences

We'll create input-output pairs of character sequences. Each input sequence will be `seq_length` characters long, and the target will be the character immediately following the input sequence.

```
seq_length = 100
step = 1 # How many characters to move forward to create new sequence

sentences = []
next_chars = []

for i in range(0, len(text) - seq_length, step):
    sentences.append(text[i: i + seq_length])
    next_chars.append(text[i + seq_length])

print(f"Number of sequences: {len(sentences)}")

# Vectorize the data
X = np.zeros((len(sentences), seq_length, len(vocab)), dtype=np.bool_)
y = np.zeros((len(sentences), len(vocab)), dtype=np.bool_)

for i, sentence in enumerate(sentences):
    for t, char in enumerate(sentence):
        X[i, t, char_to_int[char]] = 1
    y[i, char_to_int[next_chars[i]]] = 1

print(f"Shape of X: {X.shape}") # (num_sequences, seq_length, vocab_size)
print(f"Shape of y: {y.shape}") # (num_sequences, vocab_size)
```

4. Model Training: Basic Text Generation Model

We'll build a simple Recurrent Neural Network (RNN) using LSTM layers, which are effective for sequence modeling. While not a full Transformer, this model demonstrates the core principle of sequential text generation.

```
model = keras.Sequential([
    layers.LSTM(128, input_shape=(seq_length, len(vocab))),
    layers.Dense(len(vocab), activation='softmax')
```

```
])

model.summary()
```

Compile and Train the Model

```
optimizer = keras.optimizers.Adam(learning_rate=0.01)
model.compile(loss='categorical_crossentropy', optimizer=optimizer)

# Training (this might take a while depending on the text size and epochs)
epochs = 20
batch_size = 128

print("\nStarting model training...")
history = model.fit(X, y, epochs=epochs, batch_size=batch_size, verbose=1)
print("\nModel training complete.")
```

5. Application Demonstration: Text Generation

We will implement a function to generate text character by character, starting from a seed phrase. The model predicts the next character, which is then added to the sequence, and the process repeats.

```
def sample_next_char(preds, temperature=1.0):
    """
    Helper function to sample a character from a probability distribution.
    Higher temperature makes the sampling more random.
    """
    preds = np.asarray(preds).astype('float64')
    preds = np.log(preds) / temperature
    exp_preds = np.exp(preds)
    preds = exp_preds / np.sum(exp_preds)
    probas = np.random.multinomial(1, preds, 1)
    return np.argmax(probas)

def generate_text(model, seed_text, num_generate=400, temperature=0.5):
    """
    Generates text using the trained model.
    """
    generated_text = seed_text
    start_index = np.random.randint(0, len(text) - seq_length - 1)
    sentence = text[start_index: start_index + seq_length] # Take a sequence from t

    # Try to find seed_text in the corpus to start generation from a meaningful cor
    if seed_text:
        # Pad or truncate seed_text to seq_length
        if len(seed_text) < seq_length:
```

```
                sentence = ' ' * (seq_length - len(seed_text)) + seed_text
            else:
                sentence = seed_text[-seq_length:]
            generated_text = sentence

        print(f'-- Generating with seed: "{sentence}" --')

        for i in range(num_generate):
            x_pred = np.zeros((1, seq_length, len(vocab)))
            for t, char in enumerate(sentence):
                if char in char_to_int: # Ensure char is in vocab
                    x_pred[0, t, char_to_int[char]] = 1
                else:
                    # Handle unseen characters by mapping to a default (e.g., space or
                    # For this simple model, we assume all chars are in vocab after cle
                    x_pred[0, t, char_to_int[' ']] = 1 # Fallback to space

            preds = model.predict(x_pred, verbose=0)[0]
            next_index = sample_next_char(preds, temperature)
            next_char = int_to_char[next_index]

            generated_text += next_char
            sentence = sentence[1:] + next_char

        return generated_text


    # --- Practical Example: Content Creation Application ---

    print("\n--- Generating text with temperature 0.2 (more conservative) ---")
    seed_phrase_1 = "the eyes of my love"
    print(generate_text(model, seed_phrase_1, num_generate=300, temperature=0.2))

    print("\n--- Generating text with temperature 0.8 (more creative) ---")
    seed_phrase_2 = "what is love?"
    print(generate_text(model, seed_phrase_2, num_generate=300, temperature=0.8))

    print("\n--- Generating text with temperature 1.2 (more diverse/random) ---")
    seed_phrase_3 = "summer's day"
    print(generate_text(model, seed_phrase_3, num_generate=300, temperature=1.2))
```

6. Documentation and Report Writing

When preparing your 3-page report, ensure you include the following sections:

Introduction to Generative AI and its Significance

- Define Generative AI and differentiate it from discriminative AI.
- Discuss its growing importance and potential impact across industries.

Description of GPTs Architecture and its Functionality

- **Transformer Model**: Explain the core components (self-attention, feed-forward networks, positional encoding).
- **Decoder-only Structure**: Clarify why GPTs use a decoder-only architecture for generation.
- **Text Generation Process**: Detail tokenization, input embeddings, masked self-attention, probability distribution, and different sampling strategies (greedy, temperature, top-k/p).

Methodology and Findings from the Hands-on Model Implementation

- **Dataset Choice**: Describe the chosen dataset (e.g., Shakespeare's Sonnets) and why it's suitable for this task.
- **Preprocessing**: Detail steps like downloading, cleaning, lowercasing, character-to-integer mapping, and sequence creation.
- **Model Architecture**: Describe your implemented RNN/LSTM model (layers, units, input shape, output activation).
- **Training Process**: Explain the optimizer, loss function, epochs, and batch size used.
- **Findings**: Discuss the training loss curve (you can plot `history.history['loss']`). Comment on how well the model seems to learn the text structure. Analyze the quality of the generated text, noting its coherence, creativity, and any limitations observed.

Applications of Generative AI and a Practical Demonstration

- **Real-world Applications**: Discuss various applications of Generative AI beyond text generation (e.g., image generation, code generation, music composition, drug discovery, data augmentation).
- **Practical Demonstration**: Describe your text generation demonstration. Explain the role of `seed_text` and `temperature` in influencing the output. Showcase examples of generated text with different temperatures and discuss the results (e.g., how higher temperature affects creativity vs. coherence).

Discussion on Ethical Considerations and Potential Solutions

- **Misinformation and Disinformation**: How can generative models be used to create misleading content?
- **Bias and Fairness**: How can biases present in training data be amplified in generated outputs?
- **Copyright and Plagiarism**: What are the implications for original content creation?
- **Security Risks**: Discuss potential for phishing, spam, or malicious code generation.
- **Economic Impact**: How might these technologies affect job markets?

- **Potential Solutions**: Discuss approaches like content provenance, watermarking, ethical guidelines for development and deployment, bias detection/mitigation techniques, and responsible AI frameworks.

Conclusion Summarizing Key Insights and Future Perspectives in Generative AI

- Recap the key learnings from exploring GPTs and implementing a generative model.
- Discuss the current state and future potential of Generative AI, including trends like multimodal generation, smaller efficient models, and specialized applications.